
يَسْمِ اللّٰهَ الرَّحْمٰنَ الرَّحِيْمَ



Natural language Processing

Lecture – Feature Extraction from Text

Author: Dr. Rao Muhammad Adeel Nawab

Instructor: Waqar Ashiq

How to Work?

کام کرنا۔



خوشی خوشی کام کرنا۔



اللہ کو ساتھ لے کر خوشی خوشی کام کرنا۔



آیت: **إِيَّاكَ نَعْبُدُ وَإِيَّاكَ نَسْتَعِينُ**

ترجمہ: یا اللہ ہم تیری ہی عبادت کرتے ہیں۔
اور تجھ ہی سے مدد مانگتے ہیں

Power of Effort & Dua

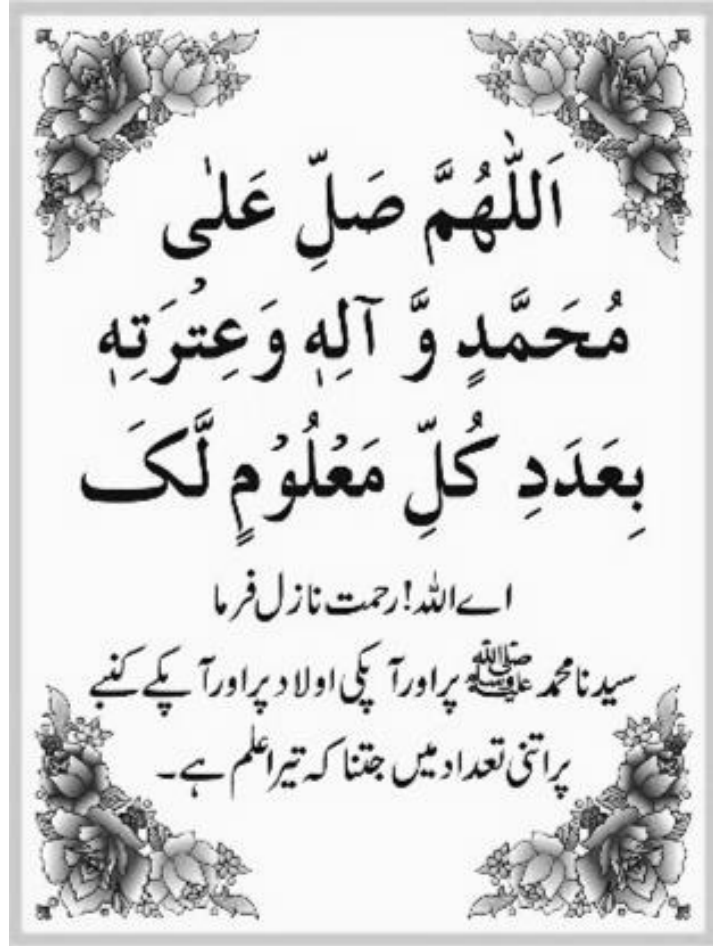
محنت کبھی نہیں ہارتی۔

دعائیں ہوں تو کھوٹے سکے بھی چل جاتے ہیں۔

دعا: اِهْدِنَا الصِّرَاطَ الْمُسْتَقِيمَ صِرَاطَ الَّذِينَ أَنْعَمْتَ عَلَيْهِمْ

ترجمہ: ہمیں سیدھی راہ دکھا ان لوگوں کی راہ جن پر تو نے انعام کیا۔

Dua – Take Help from Allah Before Starting Any Task



اللَّهُمَّ خِزْ لِي وَاخْتَرْ لِي

سُبْحَانَكَ لَا عِلْمَ لَنَا إِلَّا مَا عَلَّمْتَنَا

إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

رَبِّ اشْرَحْ لِي صَدْرِي

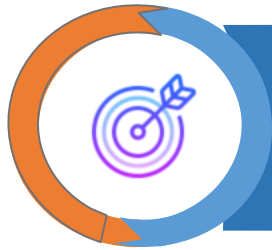
وَيَسِّرْ لِي أَمْرِي

وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي

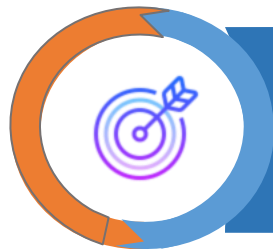
يَفْقَهُوا قَوْلِي

Course Focus

Mainly get Excellence in two things



Become a great Human Being



Become a great Machine Learning Engineer

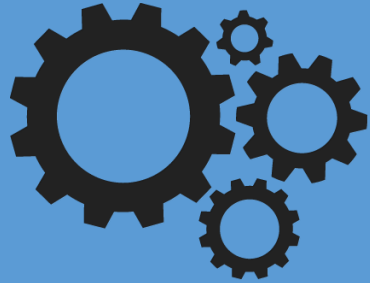
Lecture Outline

- Features in NLP / Machine Learning
- Representing Textual Features as N-grams
- Feature Extraction Examples
- Feature Extraction + Feature Selection Examples

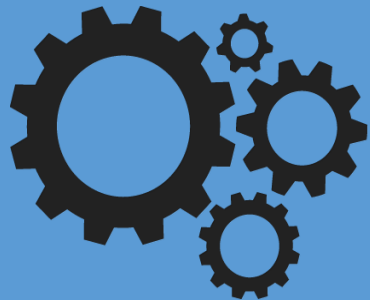


Features in NLP/ Machine Learning

Feature



A feature is an individual measurable property or characteristic of an object



Many algorithms in machine learning require a numerical representation of objects, since such representations facilitate processing and statistical analysis

Feature Engineering

○ Definition

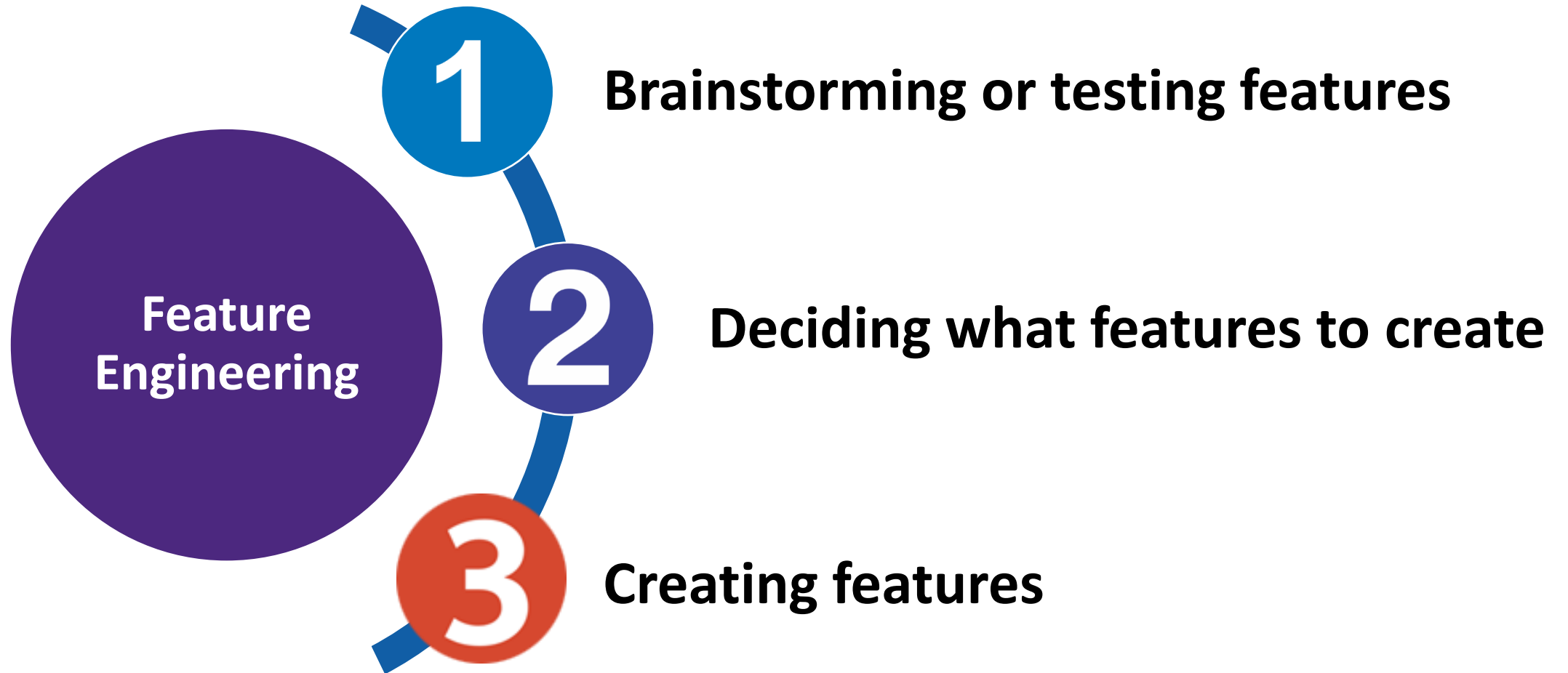
▮ Feature engineering is the process of using domain knowledge of the data to create features that make machine learning algorithms work

○ Feature engineering is fundamental to the application of machine learning and is both difficult and expensive

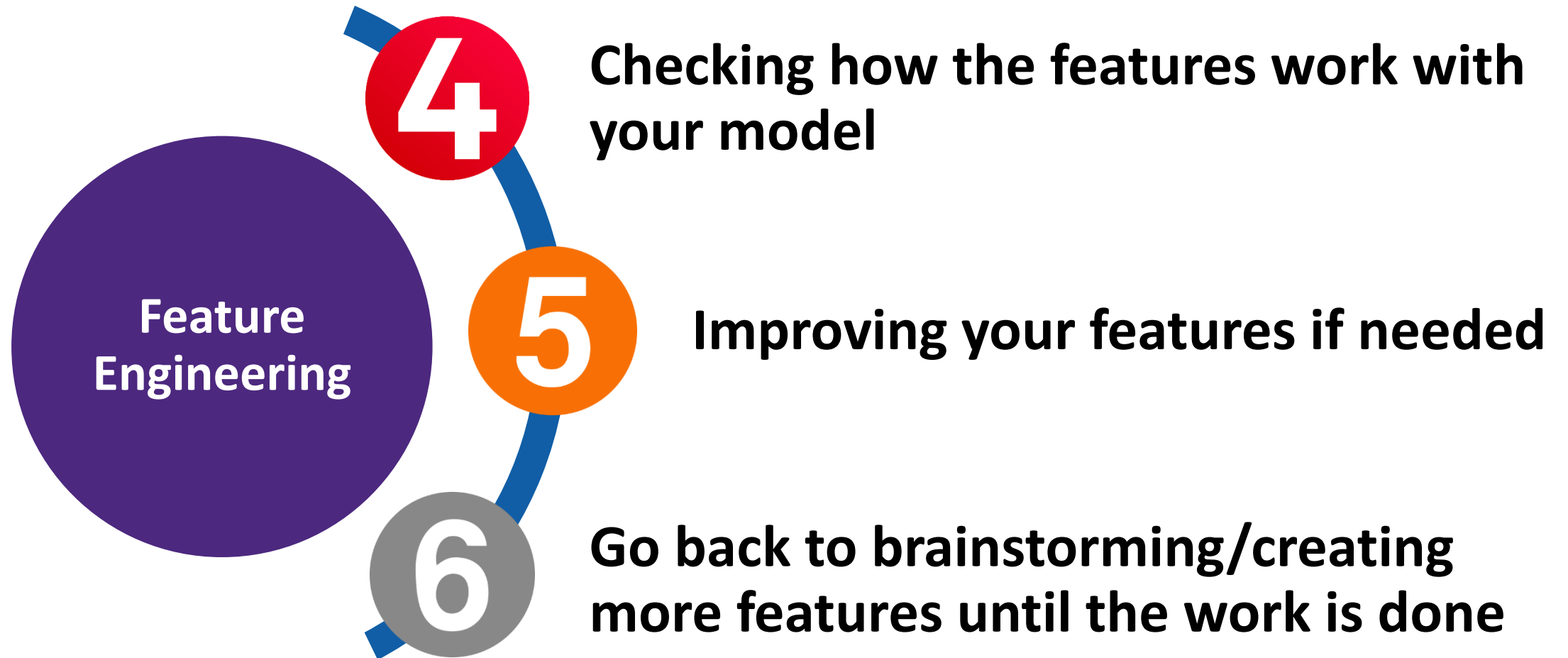
Feature Engineering (cont...)

- **Feature engineering is an informal topic, but it is considered essential in applied machine learning**
- **Coming up with features is difficult, time-consuming, requires expert knowledge**
- **The need for manual feature engineering can be obviated by automated feature learning**

Feature Engineering – Process



Feature Engineering – Process (cont...)

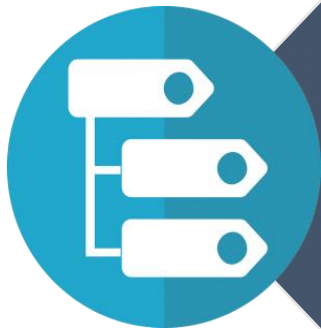


Automated Feature Learning



○ Definition

- ▮ Automated Feature learning (or representation learning) is a set of techniques that allows a system to automatically discover the representations needed for feature detection or classification from raw data



- Automated Feature Learning replaces manual feature engineering and allows a machine to both learn the features and use them to perform a specific task

Feature Extraction

- Feature Extraction Is the process of extracting “features” (or attributes) from data in a format supported by machine learning algorithms
- Data can be
 - Text
 - Image
 - Video
 - Audio
- Scikit-Learn module for feature extraction
 - `sklearn.feature_extraction`

Feature Selection

- Choosing informative, discriminating and independent features is a crucial step in machine learning
- Feature Selection
 - is the process of selecting a “subset of relevant features” from the original set of extracted features
- Goal of Feature Selection is to improve machine learning algorithms' Accuracy or to boost their performance on very high-dimensional data
- Scikit-Learn module for feature selection
 - `sklearn.feature_selection`

Feature Vector



Definition

- ▮ A feature vector is just a vector that contains information describing an object's important characteristics



More Formal Definition

- ▮ A feature vector is an n-dimensional vector of numerical features that represents some object



In Image Data

- ▮ Feature = Pixels of an Image

Feature Vector



In Text Data

- ▮ **Feature = Frequency Counts of Words in a Text**



Example - Feature Vector of an Image

- ▮ **A vector of raw intensity value of each pixel (or feature)**

Image Features = <Pixel 1, Pixel 2, Pixel 3, Pixel 4>

Image Feature Vector = <100,200, 150, 200>

Feature Vector



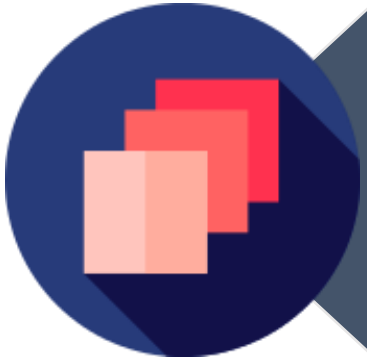
Example - Feature Vector of a Text

- A vector “frequency count” of each word (or feature) appearing in the text

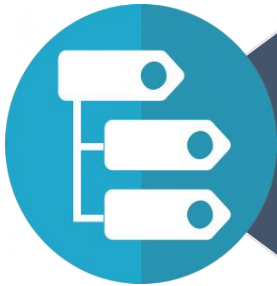
Text Features = $\langle a, \text{nice}, \text{car} \rangle$

Text Feature Vector = $\langle 1, 0, 3 \rangle$

Feature Space



- Feature space refers to the n -dimensions where your features live (not including a target attribute, if it is present)



- Each feature is a dimension in feature space

Vocabulary and Stop Words



Vocabulary

- ▮ The set of unique words (or tokens) in a corpus



Stop Words

- ▮ The set of commonly used words

Vocabulary and Stop Words



Example – Consider the following corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Vocabulary and Stop Words



Vocabulary (with stop words)

□ {a, nice, car, good, beautiful, black}



Vocabulary (without stop words)

□ {nice, car, good, beautiful, black}

Feature Representation of Textual Data



In Text

- ▮ Features are made up of words (or tokens) / characters



Transforming String (Text) to Vector

- ▮ Transform documents (or corpus or collection of texts) to document-term matrix

Feature Representation of Textual Data (cont...)

Bag of Words BOW (Weighting Features Techniques)

- Binary Weight

- Assign 1 if a feature occurs in a text document otherwise 0

- Count (or Term Frequency) Weight

- Use “frequency count” of a feature as weight

Feature Representation of Textual Data (cont...)

Weighting Features – IDF Weight

- IDF (Inverse Document Frequency) Weight

- Document Frequency

- The number of documents (in a corpus) containing a particular term

- Term Frequency Weight

- Measures how frequently a term occurs in a document

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

Feature Representation of Textual Data (cont...)

Weighting Features – IDF Weight

- Inverse Document Frequency

- ▮ Measures how important a term is

$$IDF(t) = \frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}}$$

- TF.IDF Weight

$$TFIDF(t) = TF(t) * IDF(t)$$



Representing Textual Features as N-grams

Representing Textual Features as N-grams

Aim

The main aim of this tutorial is to predict the gender of an author from his/her written text using Scikit-Learn Machine Learning toolkit

Task

- Learn Input-Output Function
- Given a Facebook comment (input) predict the gender of the author (output) who wrote it

Representing Textual Features as N-grams

Goal

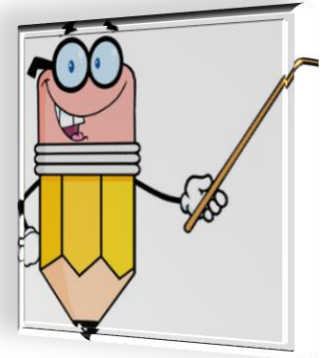
- The problem of gender prediction is treated as a supervised learning problem
- We need
 - ▢ Labelled data
 - ▢ High quality data
 - ▢ Large amount of data

Input and Output



Input - Facebook Comment

- Plain text only
- Represented as n-grams
- Note that in this tutorial we will use word uni-grams (or 1-grams) as features to represent a Facebook comment (or text)



Output - Gender of Author who wrote the comments

- Represented as Gender attribute (Male/Female)

N-gram Models

- An n-gram is a contiguous sequence of n items from a given sample of text
- N-gram can be
 - Word based
 - Character based
- N represents the length of N-gram

Example – N-gram Generation from Input Text



Input Text

▮ **R u coming?**



Word Uni-grams (N = 1)

▮ **Toeknized Text:**

R

u

coming

?

▮ **Set of Word Uni-grams = {R, u, coming, ?}**

Example – N-gram Generation from Input Text



Input Text

▮ **R u coming?**



Word Bi-grams (N = 2)

▮ **Toeknized Text:**

R

u

coming

?

▮ **Set of Word Bi-grams = {R u, u coming, coming ?}**

Example – N-gram Generation from Input Text



Input Text

▮ **R u coming?**



Word Tri-grams (N = 3)

▮ **Toeknized Text:**

R

u

coming

?

▮ **Set of Word Tri-grams = {R u coming, u coming ?}**

Example – N-gram Generation from Input Text



Input Text

□ R u coming?



Character Tri-grams (N = 3)

□ Tokenized Text: Note that **space** is also a character

R		u		C	o	m	i	n	g	?
---	--	---	--	---	---	---	---	---	---	---

□ Set of Character Tri-grams = {R u, u ,u c, co,com,omi,min,ing,ng?}



Feature Extraction Examples

Example 01 – Representing Textual Features as N-grams



Feature Extraction - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 01 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 1**
- ▮ **Max length of N = 1**



Stop Words Removal = False



Max Features = Vocabulary Size



Term Weighting = Binary

Example 01 – Representing Textual Features as N-grams



Document-Term Matrix (Binary Weights)

A	Nice	Car	Good	Beautiful	Black
1	1	1	0	0	0
1	0	0	1	0	0
1	1	1	0	1	0
1	1	1	0	0	1

Example 02 – Representing Textual Features as N-grams



Feature Extraction - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 02 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 1**
- ▮ **Max length of N = 1**



Stop Words Removal = False



Max Features = Vocabulary Size



Term Weighting = Term Frequency (tF)

Example 02 – Representing Textual Features as N-grams



Document-Term Matrix (Count (or Term Frequency) Weight)

A	Nice	Car	Good	Beautiful	Black
1	1	1	0	0	0
1	0	1	1	0	0
2	1	2	0	1	0
1	1	2	0	0	1

Example 03 – Representing Textual Features as N-grams



Feature Extraction - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 03 – Representing Textual Features as N-grams



N-gram = word

- Min length of N = 1
- Max length of N = 1

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

$$IDF(t) = \frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}}$$



Stop Words Removal = False



Max Features = Vocabulary Size



Term Weighting = (TFIDF)

Example 03 – Representing Textual Features as N-grams

$$TF(t) = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in document}}$$

$$IDF(t) = \frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}}$$

Document 1: "a nice car" (3 words)

- $TF("a") = 1/3$
- $TF("nice") = 1/3$
- $TF("car") = 1/3$

Document 2: "a good car" (3 words)

- $TF("a") = 1/3$
- $TF("good") = 1/3$
- $TF("car") = 1/3$

Document 3: "a beautiful car a nice car" (6 words)

- $TF("a") = 2/6 = 1/3$
- $TF("beautiful") = 1/6$
- $TF("car") = 2/6 = 1/3$
- $TF("nice") = 1/6$



Document 4: "a nice black car car" (5 words)

- $TF("a") = 1/5$
- $TF("nice") = 1/5$
- $TF("black") = 1/5$
- $TF("car") = 2/5$

IDF calculations:

- $IDF("a") = \log(4/4) = 0$
- $IDF("nice") = \log(4/3) \approx 0.1249$
- $IDF("car") = \log(4/4) = 0$
- $IDF("good") = \log(4/1) \approx 1.3863$
- $IDF("beautiful") = \log(4/1) \approx 1.3863$
- $IDF("black") = \log(4/1) \approx 1.3863$

Example 03 – Representing Textual Features as N-grams



Document-Term Matrix (TF-IDF Weights)

A	Nice	Car	Good	Beautiful	Black
0	0.0416	0	0	0	0
0	0	0	0.4621	0	0
0	0.0208	0	0	0.2310	0
0	0.0249	0	0	0	0.2773

Example 04 – Representing Textual Features as N-grams



Feature Extraction - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 04 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 1**
- ▮ **Max length of N = 1**



Stop Words Removal = True



Max Features = Vocabulary Size



Term Weighting = Term Frequency

Example 04 – Representing Textual Features as N-grams



Document-Term Matrix (with Stop Words Removal)

Nice	Car	Good	Beautiful	Black
1	1	0	0	0
0	0	1	0	0
1	2	0	1	0
1	2	0	0	1



Feature Extraction + Feature Selection Examples

Example 01 – Representing Textual Features as N-grams



Feature Extraction + Feature Selection - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 01 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 1**
- ▮ **Max length of N = 1**



Stop Words Removal = True



Max Features = 4



Term Weighting = Term Frequency

Example 01 – Representing Textual Features as N-grams



Document-Term Matrix

Nice	Good	Beautiful	Black
1	0	0	0
0	1	0	0
1	0	1	0
1	0	0	1

Example 02 – Representing Textual Features as N-grams



Feature Extraction + Feature Selection - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 02 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 3**
- ▮ **Max length of N = 3**



Stop Words Removal = True



Max Features = 4



Term Weighting = Term Frequency

Example 02 – Representing Textual Features as N-grams



Document-Term Matrix

Beautiful Car Nice	Black Car Car	Car Nice Car	Nice Black Car
0	0	0	0
0	0	0	0
1	1	1	0
0	1	0	1

Example 03 – Representing Textual Features as N-grams



Feature Extraction + Feature Selection - Corpus

Comment	Gender
a nice car	Female
a good car	Male
a beautiful car a nice car	Female
a nice black car car	Male

Example 03 – Representing Textual Features as N-grams



N-gram = word

- ▮ **Min length of N = 1**
- ▮ **Max length of N = 3**



Stop Words Removal = True



Max Features = 10



Term Weighting = Term Frequency

Example 03 – Representing Textual Features as N-grams



Document-Term Matrix

Beautiful	Beautiful Car	Beautiful Car Nice	Black	Black Car	Black Car Car	Car	Car Car	Nice	Nice Car
0	0	0	0	0	0	1	0	1	1
0	0	0	0	0	0	1	0	0	0
1	1	1	0	0	0	2	0	1	1
0	1	0	1	1	1	2	1	1	0